UniSENAI

Estruturas de Dados

Aula 02b: Estruturas de dados homogêneas, da teoria à excelência na prática.

Matheus Luan Krueger, M.e

UniSENAI

Roteiro

- Tipos primitivos de dados no Java
- Estruturas de dados homogêneas
  - Motivação
- Vetores (arrays)
- Matrizes (arrays bidimensionais)
- Problemas de lógica de programação em Java

UniSENAI

Tipos primitivos de dados no Java

- Muitos dos valores usados em programas baseados na tecnologia Java são armazenados como tipos de dados primitivos. Abaixo estão listados os **oito tipos primitivos** incorporados à linguagem de programação Java:

  - Tipos integrais: **byte**, **short**, **int** e **long**
  - Tipos de ponto flutuante: **float** e **double**
  - Tipo textual: **char**
  - Tipo lógico: **boolean**

UniSENAI

Tipos primitivos de dados no Java

TIPO PRIMITIVO	VALORES	OBSERVAÇÕES	BITS EM MEMÓRIA	EXEMPLO
boolean	true ou false	O valor booleano assume valores true (verdadeiro) ou false (falso)	8	true
byte	Entre -128 e 127	Números inteiros de 8 bits de precisão	8	108
char	Entre 0 e 65535	Na teoria, char armazena números inteiros de 16 bits de precisão, mas na prática, se utilizam para qualquer caractere alfanumérico (char = character)	16	s
double	Entre 4.94065645841246544e-324 e 1.7976931348623157e+308	Pontos flutuantes de precisão dupla, usados para representar valores decimais	64	34897.75
float	Entre 1.40239846e-46 e 3.40282347e+38	Pontos flutuantes de precisão simples, usados para representar valores decimais	32	34897.75
int	Entre -2.147.483.648 e +2.147.483.647	Números inteiro de 32 bits de precisão	32	1000
long	Entre -9.223.372.036.854.775.808 e +9.223.372.036.854.775.807	Números inteiros de 64 bits de precisão	64	7234829
short	Entre -32768 e 32767	Números inteiros de 16 bits de precisão	16	1000

UniSENAI

Tipos primitivos de dados no Java

- O Java fornece quatro tipos de inteiro para acomodar números de diversos tamanhos. **Todos os tipos numéricos têm sinal** e, portanto, podem comportar números positivos ou negativos. Os tipos de ponto flutuante armazenam números com uma parte fracionária e obedecem ao padrão **IEEE 754**.

Inteiro	Ponto Flutuante	Caractere	Verdadeiro Falso
byte short int long	float double	char	boolean
0	**0.0**	**'\u0000'**	**false**

UniSENAI

Estruturas de dados homogêneas

- Representar **diversas** variáveis associadas:

```
public class NomeDaClasse {

	public static void main(String args[]){

		// números para um bilhete de loteria
		int numLoteria1;
		int numLoteria2;
		int numLoteria3;
		int numLoteria4;
		int numLoteria5;
		...
		// algumas loterias demandam 15 números... devo ter 15 variáveis separadas?
	}
}
```

UniSENAI

Estruturas de dados homogêneas: vetores

- **@DEVMEDIA**
  - Vetores são estruturas de dados que armazenam usualmente uma quantidade fixa de dados de um certo tipo; por esta razão, também são conhecidos como estruturas homogêneas de dados.
  - Internamente, um vetor armazena diversos valores, cada um associado a um número que se refere à **posição de armazenamento**, e é conhecido como **índice**. Os vetores são estruturas indexadas, em que cada valor que pode ser armazenado em uma certa posição (índice) é chamado de elemento do vetor

```
tipo[] identificador [= alocação de espaço] ;
tipo identificador[] [= alocação de espaço] ;

A alocação de espaço pode ser feita na própria linha da criação do vetor, ou posteriormente. Exemplo:

      int vetor[];
      vetor = new int[10];

      int outroVetor = new int[10];
```

UniSENAI

Estruturas de dados homogêneas: vetores

- **Sintaxe** herdada de outras linguagens (C)
- Vetores são objetos, o que significa que no exemplo abaixo temos apenas referências:

```
int vetor[];
int[] outroVetor;
```

- Para criar/instanciar estes objetos vetores, utilizamos a seguinte sintaxe:

```
vetor = new int[5];          // alocando 5  espaços
outroVetor = new int[15];    // alocando 15 espaços
```

UniSENAI

Estruturas de dados homogêneas: vetores

- Também podemos **criar/instanciar vetores** de forma dinâmica, isto é, generalizar o tamanho do vetor:

```
Scanner leitor = new Scanner(System.in);
System.out.println("Informe um tamanho para o vetor"):
int tamanho = leitor.nextInt();

int vetor = new int[tamanho];

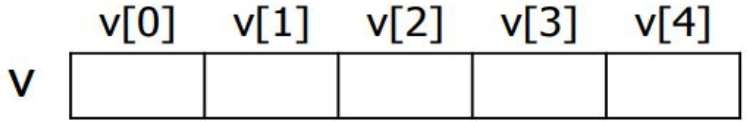
System.out.println("Tamanho do vetor: " + vetor.length);
```

- Podemos utilizar o atributo **length** para verificar o real tamanho

UniSENAI

Estruturas de dados homogêneas: vetores

- Em Java, vetores têm **tamanho fixo** e, uma vez criados, **não podem ser redimensionados**.
- Um vetor é um objeto da classe Object. Uma vez criado, este objeto possui apenas o campo length (tamanho do vetor) e somente os métodos da classe Object.
- Como para qualquer objeto, apenas a declaração do vetor não cria o objeto. A declaração cria apenas uma variável de referência (ponteiro) para o vetor. A **memória a ser utilizada pelo vetor é alocada pela instrução new** ou pela inicialização do vetor.
- Os elementos de um vetor são identificados por meio de **índices**. O primeiro elemento tem o índice 0 (zero). Portanto, para um vetor de n elementos, os índices variam de **0 a n-1**.




```
int v[];
v = new int[5];
```

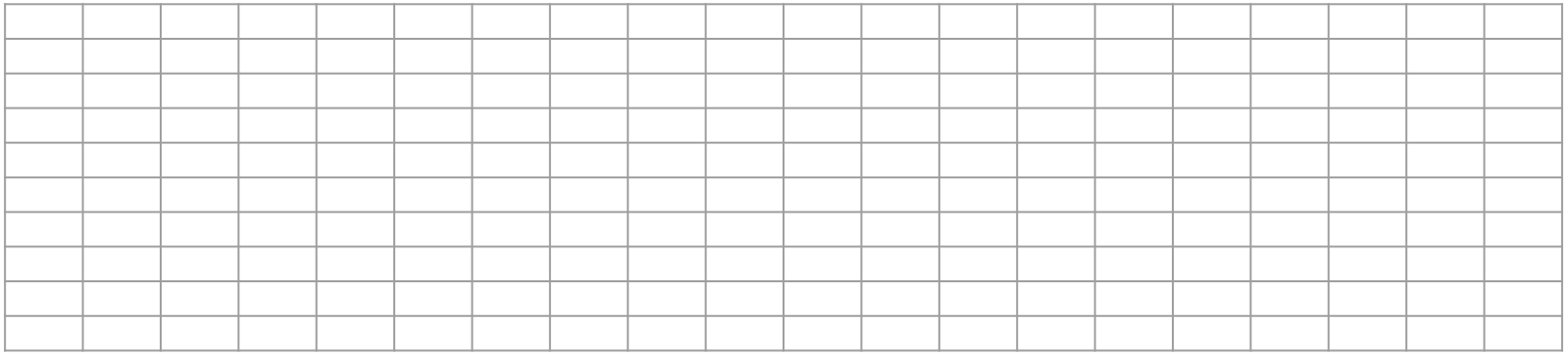
UniSENAI

Estruturas de dados homogêneas: vetores

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?

```
int v[];
v = new int[5];
```

  - v é declarado como um vetor de inteiros
  - a expressão new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v



⬜ = 1 byte

UniSENAI

Estruturas de dados homogêneas: vetores

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?

```
int v[];
v = new int[5];
```

  - v é declarado como um vetor de inteiros
  - a expressão new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

Estruturas de

A partir da execução da primeira linha, aloca-se, na tabela de execução do programa, uma nova variável (do tipo vetor), com sua referência apontando para NULL.

UniSENAI

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?
  - v é declarado como um vetor de inteiros
  - a expressão new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

```
int v[];   está apontando para NULL
v = new int[5];
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

A segunda linha é resolvida em duas etapas. A primeira (new int[5]) diz respeito à alocação do espaço destinado para 5 variáveis do tipo inteiro... O tipo inteiro ocupa 32 bits, isto é, 4 bytes na memória. Portanto, faz-se necessário alocar 5 * 4 bytes (20 bytes) na memória.

UniSENAI

Estruturas de

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?
  - v é declarado como um vetor de inteiros
  - a expressão new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

```
int v[];
v = new int[5];
```
necessário alocar espaço

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

▢ = 1 byte

A segunda linha é resolvida em duas etapas. A primeira (new int[5]) diz respeito à alocação do espaço destinado para 5 variáveis do tipo inteiro... O tipo inteiro ocupa 32 bits, isto é, 4 bytes na memória. Portanto, faz-se necessário alocar 5 * 4 bytes (20 bytes) na memória.

UniSENAI

Estruturas de

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?
  - v é declarado como um vetor de inteiros
  - new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

```
int v[];
v = new int[5];  necessário alocar espaço
```

Note que podem existir dados que já estejam ocupando a memória...

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

A segunda linha é resolvida em duas etapas. A primeira (new int[5]) diz respeito à alocação do espaço destinado para 5 variáveis do tipo inteiro... O tipo inteiro ocupa 32 bits, isto é, 4 bytes na memória. Portanto, faz-se necessário alocar 5 * 4 bytes (20 bytes) na memória.

UniSENAI

Estruturas de

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?
  - v é declarado como um vetor de inteiros

```
int v[];
v = new int[5];
```

Encontramos nossos 20 bytes na memória... Note que cada quadrado representa o espaço de 8 bits (1 byte).

cria efetivamente um vetor de inteiros, de tamanho 5

o (sinal de igual) associa o vetor criado à referência do vetor v

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

= 1 byte

Estruturas de

A segunda etapa prevista na linha 2 trata do sinal de atribuição (=). É por meio dessa atribuição que o programa compreenderá que a referência do vetor v deve 'apontar' para o endereço de memória inicial 08

UniSENAI

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?
  - v é declarado como um vetor de inteiros
  - a expressão new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

```
int v[];
v = new int[5];
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

E é dessa forma que os índices utilizados em vetores começam a fazer sentido! Observe que na verdade, os 20 bytes reservados são:

4 bytes (32 bits) + 4 bytes (32 bits) + 4 bytes (32 bits) + 4 bytes (32 bits) + 4 bytes (32 bits)

Estruturas de

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições do vetor?
  - v é declarado como um vetor de inteiros
  - a expressão new int[10] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

```
int v[];
v = new int[5];
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

= 1 byte

Como acessar vetores? Por meio de índices.

- Os elementos de um vetor são identificados por meio de **índices**. O primeiro elemento tem o índice 0 (zero). Portanto, para um vetor de n elementos, os índices variam de **0 a n-1**.

Estruturas de

- Mas e quando é, de fato, alocada? Quais s posições do vetor?
  - v é declarado como um vetor de inteiros
  - a expressão new int[5] cria efetivamente um vetor de inteiros, de tamanho 5
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

```
int v[];
v = new int[5];
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

= 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal [illegible]gual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

= 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal [illegible]gual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

Posição hexadecimal 08 representa 8 em decimal, portanto 8 quadrados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A																	
3C	3D	3E																	
50	51	52																	
64	65	66																	
78	79	7A																	
8C	8D	8E																	
A0	A1	A2																	
B4	B5	B6																	

Para acessarmos as posições 0, 1, 2, 3 e 4
(note que a posição 4 é n-1, onde n representa 5 elementos/posições do vetor)

⬜ = 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal igual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

Posição hexadecimal 08 representa 8 em decimal, portanto 8 quadrados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A																	
3C	3D	3E																	
50	51	52																	
64	65	66																	
78	79	7A																	
8C	8D	8E																	
A0	A1	A2																	
B4	B5	B6																	

Para acessarmos as posições 0, 1, 2, 3 e 4
(note que a posição 4 é n-1, onde n representa 5 elementos/posições do vetor)

```
0 * 32 + 64 = 64        =>  64 bits / 8 (1 ⬜ = 8 bits) = 08 convertendo para HEX =>   08
```

⬜ = 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

Posição hexadecimal 08 representa 8 em decimal, portanto 8 quadrados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A																	
3C	3D	3E																	
50	51	52																	
64	65	66																	
78	79	7A																	
8C	8D	8E																	
A0	A1	A2																	
B4	B5	B6																	

Para acessarmos as posições 0, 1, 2, 3 e 4
(note que a posição 4 é n-1, onde n representa 5 elementos/posições do vetor)

```
0 * 32 + 64 = 64        =>  64 bits / 8 (1 ⬜ = 8 bits) = 08 convertendo para HEX =>  08
1 * 32 + 64 = 96        =>  96 bits / 8 (1 ⬜ = 8 bits) = 12 convertendo para HEX =>  0C
```

⬜ = 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

Posição hexadecimal 08 representa 8 em decimal, portanto 8 quadrados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A																	
3C	3D	3E																	
50	51	52																	
64	65	66																	
78	79	7A																	
8C	8D	8E																	
A0	A1	A2																	
B4	B5	B6																	

Para acessarmos as posições 0, 1, 2, 3 e 4
(note que a posição 4 é n-1, onde n representa 5 elementos/posições do vetor)

```
0 * 32 + 64 = 64      =>  64 bits / 8 (1 ⬜ = 8 bits) = 08 convertendo para HEX =>  08
1 * 32 + 64 = 96      =>  96 bits / 8 (1 ⬜ = 8 bits) = 12 convertendo para HEX =>  0C
2 * 32 + 64 = 128     => 128 bits / 8 (1 ⬜ = 8 bits) = 16 convertendo para HEX =>  10
```

⬜ = 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal igual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

Posição hexadecimal 08 representa 8 em decimal, portanto 8 quadrados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A																	
3C	3D	3E																	
50	51	52																	
64	65	66																	
78	79	7A																	
8C	8D	8E																	
A0	A1	A2																	
B4	B5	B6																	

Para acessarmos as posições 0, 1, 2, 3 e 4
(note que a posição 4 é n-1, onde n representa 5 elementos/posições do vetor)

```
0 * 32 + 64 = 64      =>  64 bits / 8 (1 ⬜ = 8 bits) = 08 convertendo para HEX =>  08
1 * 32 + 64 = 96      =>  96 bits / 8 (1 ⬜ = 8 bits) = 12 convertendo para HEX =>  0C
2 * 32 + 64 = 128     => 128 bits / 8 (1 ⬜ = 8 bits) = 16 convertendo para HEX =>  10
3 * 32 + 64 = 160     => 160 bits / 8 (1 ⬜ = 8 bits) = 20 convertendo para HEX =>  14
```

⬜ = 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  do vetor?
  - v é declarado
  - a expressão ne
  - o comando de atribuição (sinal de igual) associa o vetor criado à referência do vetor v

Como assim? De 0 à n-1?

Qual o endereço inicial de memória que o vetor 'v' foi associado? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de vetores realiza o cálculo de OFFSET !

Posição hexadecimal 08 representa 8 em decimal, portanto 8 quadrados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A																	
3C	3D	3E																	
50	51	52																	
64	65	66																	
78	79	7A																	
8C	8D	8E																	
A0	A1	A2																	
B4	B5	B6																	

= 1 byte

Para acessarmos as posições 0, 1, 2, 3 e 4
(note que a posição 4 é n-1, onde n representa 5 elementos/posições do vetor)

```
0 * 32 + 64 = 64     =>  64 bits / 8 (1 ⬜ = 8 bits) = 08 convertendo para HEX =>  08
1 * 32 + 64 = 96     =>  96 bits / 8 (1 ⬜ = 8 bits) = 12 convertendo para HEX =>  0C
2 * 32 + 64 = 128    => 128 bits / 8 (1 ⬜ = 8 bits) = 16 convertendo para HEX =>  10
3 * 32 + 64 = 160    => 160 bits / 8 (1 ⬜ = 8 bits) = 20 convertendo para HEX =>  14
4 * 32 + 64 = 192    => 192 bits / 8 (1 ⬜ = 8 bits) = 24 convertendo para HEX =>  18
```

UniSENAI

Estruturas de dados homogêneas: vetores

- `[]` é o "operador de indexação"
- Para acessar/atribuir valores sempre devemos utilizá-lo
- Acessos ilegais geram exceções em tempo de execução
  - Exemplos de acessos ilegais:
    - índices negativos
    - índices de posições que não existem (vetor com cinco posições, tentar acessar [5])

```
int vetor[] = new int[5];
vetor[0] = 4;                    // vetor[-1] = 55; gerará exceção!
vetor[1] = 8;
vetor[2] = 15;
vetor[3] = 16;
vetor[4] = 42;                   // vetor[5]  = 10; gerará exceção!
```

UniSENAI

Estruturas de dados homogêneas: vetores

Inteiro	Ponto Flutuante	Caractere	Verdadeiro Falso
byte short int long	float double	char	boolean
0	0.0	'\u0000'	false

- Vetores são “zerados” na criação
- Considera os valores padrão dos tipos primitivos
- Vetores **não estão limitados a tipos primitivos**
- Um vetor pode ser criado a partir de uma lista de valores

```
int vetor[] = new int[5];              // { 0, 0, 0, 0, 0 }

String[] nomes = new String[3];        // { null, null, null }

int   primos[] = { 2, 3, 5, 7, 11 };                          // length = 5
char   diass[] = { 'd', 's', 't', 'q', 'q', 's', 's' };        // length = 7
String meses[] = { "jan", "fev, "mar", "abr", "mai", "jun" };// length = 6
```

UniSENAI

Estruturas de dados homogêneas: vetores

- O processo de **iteração** significa percorrer todos os seus elementos
- Normalmente utilizamos uma **estrutura de repetição** para tal processo

```
int   primos[] = { 2, 3, 5, 7, 11 };

for (int i = 0 ; i < primos.length ; i++) {

     int primoAtual = primos[i];
     System.out.println(primoAtual);
}
```

UniSENAI

Estruturas de dados homogêneas: vetores

- O processo de **iteração** significa percorrer todos os seus elementos
- Também podemos utilizar uma sintaxe mais interessante (*enhanced for* ou **for-each**):

```
int   primos[] = { 2, 3, 5, 7, 11 };

for (int primoAtual : primos) {

     System.out.println(primoAtual);
}
```

UniSENAI

Estruturas de dados homogêneas: vetores

- **Reforçando:** os identificadores de variáveis são apenas **referências**!
- Tendo declarado uma variável (indicando que é vetor fazendo o uso de []), esta poderá ser associada a qualquer vetor de inteiros. Exemplo:

```
int[] valores10 = new int[10];
int[] valores20 = new int[20];
int i = 20;
int[] v;

switch (i) {
     case 10:      v = valores10;          break;
     case 20:      v = valores20;          break;
     default:      v = new int[30];
}
```

UniSENAI

Problemas de lógica de programação em Java

- **Vetores**
  a. Desenvolva um programa que realize a leitura de 5 valores inteiros a partir do teclado e armazene-os em um vetor (identificado como `valoresOriginais`). Após isso, deve-se apresentar mensagens que informam qual foi:
     - a média dos valores informados
     - o maior valor informado
     - o menor valor informado
     - quantos valores negativos foram informados
     - quantos valores ímpares foram informados

  b. Desenvolva um programa que realize a leitura de 8 valores inteiros a partir do teclado e armazene-os em um vetor (identificado como `valoresOriginais`). Após isso, deve-se criar um novo vetor com o mesmo tamanho do anterior. Os valores informados pelo usuário devem ser copiados ao novo vetor de forma que fiquem dispostos em ordem inversa ao original. Por fim, deve-se mostrar na tela os valores contidos em ambos os vetores.

  c. Desenvolva um programa que realize a leitura de 8 valores inteiros a partir do teclado e armazene-os em um vetor (identificado como `valoresOriginais`). Solicite um valor a ser buscado dentro do vetor ao usuário e contabilize quantas vezes o valor está presente do vetor. Ao final, apresente a contabilização ao utilizador do programa.

  d. Desenvolva um programa que realize a leitura de 10 valores char a partir do teclado e armazene-os em um vetor (identificado como `caracteresInformados`). Solicite um caractere a ser buscado dentro do vetor ao usuário e apresente uma mensagem informando se o caractere informado existe ou não dentro do vetor.

  e. Desenvolva um programa que realize a leitura de 5 valores inteiros a partir do teclado e armazene-os em um vetor (identificado como `valoresOriginais`). Após isso, deve-se criar um novo vetor com o triplo do tamanho do anterior. Os valores informados pelo usuário deverão ser replicados nas posições 1, 4, 7, 10, 13 deste novo vetor (identificado como `valoresFinais`). As demais posições deste novo vetor deverão ser preenchidas com os valores que antecedem e sucedem os valores informados pelo usuário. Ao final deve-se mostrar na tela os valores contidos em ambos os vetores.

UniSENAI

Estruturas de dados homogêneas: matrizes

- **@DEVMEDIA**
  - Segundo DEITEL (2005, pág. 204), um array é um grupo de variáveis (elementos ou componentes) que contém valores que são todos do mesmo tipo (estruturas homogêneas de dados). Em Java os arrays são objetos, portanto são considerados tipos por referência. Internamente, o que em geral considera-se um array é, na verdade, uma referência a um objeto array na memória. **Pode-se encontrar como sinônimos para arrays: variáveis indexadas, variáveis subscritas, tabela de dados em memória ou matrizes (arrays bidimensionais)**.
  - Em variáveis (ou objetos) do tipo array podem ser armazenados um grupo de valores, cada um associado a um número ou números (no caso de estruturas **multidimensionais**) que se referem à posição de armazenamento. As posições do valor são conhecidas como índices ou subscritos do valor. Portanto, os arrays são estruturas indexadas, em que cada valor que pode ser armazenado em certa posição (índice) é chamado de elemento do array. **Matrizes são estruturas com indexação dupla.**

```
tipo[][] identificador [= alocação de espaço] ;
tipo identificador[][] [= alocação de espaço] ;

A alocação de espaço pode ser feita na própria linha da criação do vetor, ou posteriormente. Exemplo:

      int matriz[][];
      matriz = new int[2][4];             // uma matriz de 2 linhas e 4 colunas

      int outraMatriz = new int[9][9];    // uma matriz de 9 linhas e 9 colunas
```

UniSENAI

Estruturas de dados homogêneas: matrizes

- **Sintaxe** herdada de outras linguagens (C)
- Matrizes também são objetos, o que significa que no exemplo abaixo temos apenas referências:

```
int matriz[][];
int[][] outroExemploDeMatriz;
```

- Para criar/instanciar estes objetos matrizes, utilizamos a seguinte sintaxe:

```
vetor = new int[5][2];              // alocando 10 espaços: 5 linhas com 2 colunas
outroVetor = new int[5][15];        // alocando 75 espaços: 5 linhas com 15
colunas
```

UniSENAI

Estruturas de dados homogêneas: matrizes

- Também podemos **criar/instanciar matrizes** de forma dinâmica, isto é, generalizar o seu tamanho:

```
Scanner leitor = new Scanner(System.in);
System.out.println("Informe um tamanho para a matriz:");
int qtLin  = leitor.nextInt();
int qtCol  = leitor.nextInt();
int matriz = new int[qtLin][qtCol];

System.out.println("Qt linhas: " +matriz.length);
System.out.println("Qt colunas da 1ª linha: " +matriz[0].length);
```

- Podemos utilizar o atributo **length** para verificar o real tamanho

UniSENAI

Estruturas de dados homogêneas: matrizes

- Em Java, matrizes também têm **tamanho fixo** e, uma vez criadas, **não podem ser redimensionadas**.
- Uma matriz é um objeto da classe Object. Uma vez criado, este objeto possui apenas o campo length (tamanho) e somente os métodos da classe Object.
- Como para qualquer objeto, apenas a declaração da matriz não cria o objeto. A declaração cria apenas uma variável de referência (ponteiro) para a matriz. A **memória a ser utilizada pela matriz é alocada pela instrução new** ou pela sua inicialização.
- **Matrizes são vetores de vetores, então é possível criar uma matriz cujas linhas tenham tamanhos diferentes!**
- Os elementos de uma matriz são identificados por meio de **índices (dupla indexação)**. O primeiro índice representa o número de linhas (ou ainda qual o vetor a ser acessado), e o segundo índice representa o número de colunas (ou também qual o índice a ser acessado dentro daquele vetor). Conforme o uso mais "*comum/padrão*", **ambos variam de 0 à n-1.**

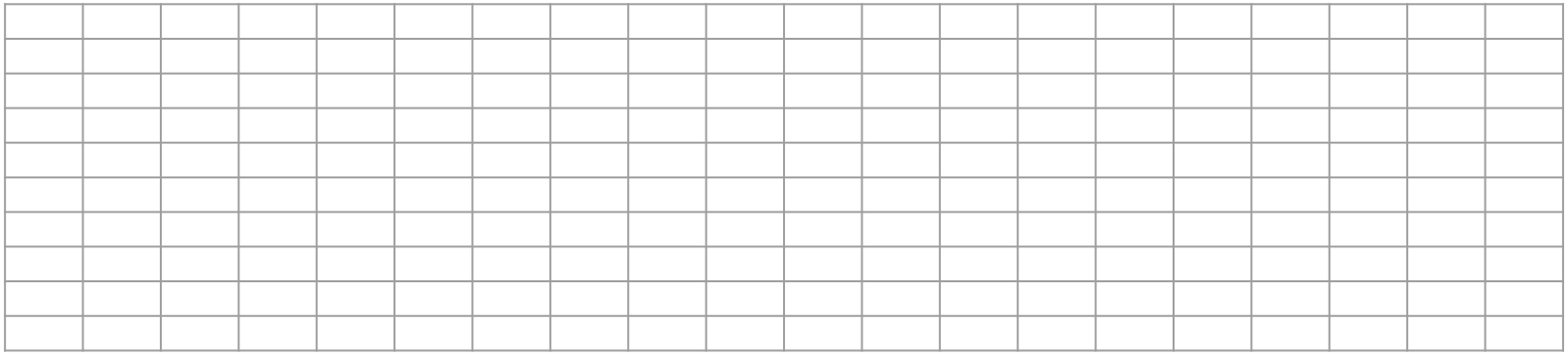
UniSENAI

Estruturas de dados homogêneas: matrizes

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?

```
int m[][];
m = new int[3][5];
```

  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'



⬜ = 1 byte

UniSENAI

Estruturas de dados homogêneas: matrizes

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?

```
int m[][];
m = new int[3][5];
```

  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

Estruturas de

A partir da execução da primeira linha, aloca-se, na tabela de execução do programa, uma nova variável (matriz), com sua referência apontando para NULL.

UniSENAI

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?

```
int m[][];   está apontando para NULL
m = new int[3][5];
```

  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

A segunda linha é resolvida em duas etapas. A primeira (new int[3][5]) diz respeito à alocação do espaço destinado para 15 variáveis do tipo inteiro... O tipo inteiro ocupa 32 bits, isto é, 4 bytes na memória. Portanto, faz-se necessário alocar 15 * 4 bytes (60 bytes) na memória.

UniSENAI

Estruturas de

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?
  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

```
int m[][];   está apontando para NULL
m = new int[3][5];   necessário alocar espaço
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

A segunda linha é resolvida em duas etapas. A primeira (new int[3][5]) diz respeito à alocação do espaço destinado para 15 variáveis do tipo inteiro... O tipo inteiro ocupa 32 bits, isto é, 4 bytes na memória. Portanto, faz-se necessário alocar 15 * 4 bytes (60 bytes) na memória.

UniSENAI

Estruturas de

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?
  - m é declarado como uma matriz de inteiros
  - new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

```
int m[][];   está apontando para NULL
m = new int[3][5];   necessário alocar espaço
```

Note que podem existir dados que já estejam ocupando a memória...

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

UniSENAI

Estruturas de

A segunda linha é resolvida em duas etapas. A primeira (new int[3][5]) diz respeito à alocação do espaço destinado para 15 variáveis do tipo inteiro... O tipo inteiro ocupa 32 bits, isto é, 4 bytes na memória. Portanto, faz-se necessário alocar 15 * 4 bytes (60 bytes) na memória.

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?
  - m é declarado como uma matriz de inteiros
  - ...] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - ...o (sinal de igual) associa a matriz criada à referência dela 'm'

```
int m[][];  está apontando para NULL
m = new int[3][5];  necessário alocar espaço
```

Encontramos nossos 60 bytes na memória... Note que cada quadrado representa o espaço de 8 bits (1 byte).

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

▢ = 1 byte

Estruturas de

A segunda etapa prevista na linha 2 trata do sinal de atribuição (=). É por meio dessa atribuição que o programa compreenderá que a referência da matriz m deve 'apontar' para o endereço de memória inicial 08

UniSENAI

- Mas e quando a instrução **new** é utilizada, como a memória é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?
  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

```
int m[][];   está apontando para NULL
m = new int[3][5];   necessário alocar espaço
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

E é dessa forma que os índices utilizados em matrizes começam a fazer sentido! Observe que na verdade, os 60 bytes reservados são:

3 linhas * (

4 bytes (32 bits) + 4 bytes (32 bits) + 4 bytes (32 bits) + 4 bytes (32 bits) + 4 bytes (32 bits)

)

Estruturas

- Mas e quan é, de fato, alocada? Quais os valores padrão assumidos em cada uma das posições da matriz?
  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

```
int m[][];   está apontando para NULL
m = new int[3][5];   necessário alocar espaço
```

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

= 1 byte

Estruturas de

Como acessar elementos de matrizes? Por meio de índices.

- Os elementos de uma matriz são identificados por meio de **índices (dupla indexação)**, sendo que a contabilização inicia no 0 (zero). Portanto, para uma matriz de n linhas e m colunas, os índices variam de **0 à n-1** e **0 à m-1**.

- Mas e quando alocada? Quais da matriz?
  - m é declarado como uma matriz de inteiros
  - a expressão new int[3][5] cria efetivamente uma matriz de inteiros, com 3 lin. com 5 col.
  - o comando de atribuição (sinal de igual) associa a matriz criada à referência dela 'm'

é, de fato, s posições

do para NULL

```
m = new int[3][5];
```
necessário alocar espaço

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

= 1 byte

Estruturas de

- Mas e quando
  alocada? Quais
  da matriz?
  - m é declarado
  - a expressão ne
  - o comando de

Como assim? De 0 à n-1? De 0 à m-1?

Qual o endereço inicial de memória que a matriz 'm' foi associada? 08
Qual o tipo de variável que foi definido na linha 1? int
Qual o tamanho (em bits) do tipo primitivo int? 32

Note que, para armazenarmos valores na memória, precisamos indicar exatamente ao computador onde (e o que) armazenar... Portanto, a indexação de matrizes realiza o cálculo de OFFSET!

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

Como assim? De 0 à n-1? De 0 à m-1?

Para acessarmos as linhas 0, 1 e 2, e suas respectivas colunas 0, 1, 2, 3 e 4
(note que a posição 2 é n-1 e 4 é m-1, onde n representa 3 linhas e m representa 5 elementos/colunas)

Linha 0, Coluna 0
64 + [0 * (5*32)] + [0 * 32] => 64 bits / 8 (1 ⬜ =8bits) = 08 conv. para **HEX =>** **08**

Linha 0, Coluna 1
64 + [0 * (5*32)] + [1 * 32] => 96 bits / 8 (1 ⬜ =8bits) = 12 conv. para **HEX =>** **0C**

Linha 0, Coluna 2
64 + [0 * (5*32)] + [2 * 32] => 128 bits / 8 (1 ⬜ =8bits) = 16 conv. para **HEX =>** **10**

Linha 0, Coluna 3
64 + [0 * (5*32)] + [3 * 32] => 160 bits / 8 (1 ⬜ =8bits) = 20 conv. para **HEX =>** **14**

Linha 0, Coluna 4
64 + [0 * (5*32)] + [4 * 32] => 192 bits / 8 (1 ⬜ =8bits) = 24 conv. para **HEX =>** **18**

Linha 2, Coluna 4
64 + [2 * (5*32)] + [4 * 32] => 512 bits / 8 (1 ⬜ =8bits) = 64 conv. para **HEX =>** **40**

iada? **08**
int
32

ecisamos indicar
anto, a indexação

ar

ados X 8 bits = 64

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
28	29	2A	2B	2C	2D	2E	2F	30	31	32	33	34	35	36	37	38	39	3A	3B
3C	3D	3E	3F	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63
64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77
78	79	7A	7B	7C	7D	7E	7F	80	81	82	83	84	85	86	87	88	89	8A	8B
8C	8D	8E	8F	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2	B3
B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3	C4	C5	C6	C7

⬜ = 1 byte

UniSENAI

Estruturas de dados homogêneas: matrizes

- `[]` é o "operador de indexação" e deve ser usado de forma dupla
- Para acessar/atribuir valores sempre devemos utilizá-lo
- Acessos ilegais geram exceções em tempo de execução
  - Exemplos de acessos ilegais:
    - índices negativos
    - índices de posições que não existem (vetor com cinco posições, tentar acessar [5])

```
int matriz[] = new int[1][3];
matriz[0][0] = 22;
matriz[0][1] = 6;
matriz[0][2] = -9;
matriz[1][0] = 11; // gerará exceção!
matriz[1][1] = 4;  // gerará exceção!
```

UniSENAI

Estruturas de dados homogêneas: matrizes

Inteiro	Ponto Flutuante	Caractere	Verdadeiro Falso
byte short int long	float double	char	boolean
0	0.0	'\u0000'	false

- Matrizes também são “zeradas” na criação
- Considera os valores padrão dos tipos primitivos
- Arrays bidimensionais **não estão limitados a tipos primitivos**
- Uma matriz pode ser criada a partir de uma lista de valores

```
1  public class MyClass {
2
3      public static void main(String args[]) {
4
5        int matriz[][] = { {1,2,3},{4,5,6,4},{9,9,9},{8,8,8} };
6
7        for(int i = 0; i < matriz.length; i++) {
8
9            System.out.println("linha");
10           for(int j = 0; j < matriz[i].length; j++) {
11             System.out.println(matriz[i][j]);
12           }
13       }
14
15   }
16 }
```

```
linha
1
2
3
linha
4
5
6
4
linha
9
9
9
linha
8
8
8
```

CPU Time: 0.04 sec(s) | Memory: 37596 kilobyte(s) | Compiled and & executed in 1.351 sec(s)

UniSENAI

Estruturas de dados homogêneas: matrizes

- O processo de **iteração** significa percorrer todos os seus elementos
- Normalmente utilizamos uma **estrutura de repetição** para tal processo

```
int matriz[][] = { {1,2,3}, {4,5,6,4}, {9,9,9}, {8,8,8} };

for(int i = 0; i < matriz.length; i++) {
     System.out.println("linha");

     for(int j = 0; j < matriz[i].length; j++) {
          System.out.println(matriz[i][j]);
     }
}
```

UniSENAI

Estruturas de dados homogêneas: matrizes

- O processo de **iteração** significa percorrer todos os seus elementos
- Também podemos utilizar uma sintaxe mais interessante (*enhanced for* ou **for-each**):

```
int matriz[][] = { {1,2,3},{4,5,6,4},{9,9,9},{8,8,8} };

for (int[] linha : matriz) {

     for (int valor : linha) {
          System.out.println(valor);
     }
}
```

UniSENAI

Problemas de lógica de programação em Java

- **Matrizes**
  a. Desenvolva um programa que soma duas matrizes 3x3 (valores inteiros), sendo que os valores de cada uma das matrizes foi solicitado ao usuário. Ao final, mostre a matriz resultante.
  b. Desenvolva um programa que solicite valores ao usuário para popular uma matriz 4x4. Feito isso, verifique se uma matriz é simétrica (igual à sua transposta).
  c. Desenvolva um programa que solicite valores ao usuário para popular uma matriz 4x4. Após isso, deve-se apresentar mensagens que informam qual foi:
     - a média dos valores armazenados na matriz
     - a soma de cada uma das linhas
     - o menor valor em cada uma das colunas
  d. Uma matriz pode ser associada com a representação de uma imagem em preto e branco, de tamanho MxN, cujos elementos assumem valores do conjunto A = {0, 1}. Dado um padrão, representado por uma matriz 3x3 também assumindo valores do conjunto A, desenvolva um programa que determina se o padrão existe ou não na "imagem". Crie uma matriz do tipo primitivo int de tamanho MxN (os valores podem ser solicitados ou definidos no código). Deve-se popular essa matriz com valores zeros e uns (pode-se atribuir aleatoriamente ou ser definido posição por posição). Solicitar um padrão 3x3, conforme ilustrado e implementar a lógica para identificar quantas vezes o padrão 3x3 aparece na "imagem/matriz".

0	0	1	0	0	0	0	1	0
0	1	1	1	1	1	1	0	0
0	0	1	0	1	1	1	0	0
0	0	0	0	0	1	0	0	0
0	0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	0	0
1	1	1	0	1	0	1	0	0
1	0	1	0	0	0	0	0	0
1	1	1	0	0	1	1	1	0

0	1	0
1	1	1
0	1	0

UniSENAI

Oportunidade - English Online 2026 - British Council

- **Que tal começar o ano batendo a sua meta do ano de estudar Inglês?** 🗺️📚
- Se você está procurando fortalecer seu inglês e melhorar suas perspectivas de empregabilidade, o Banco Santander e o British Council estão oferecendo 10.000 vagas para que você possa fazer isso de casa e no seu próprio ritmo, durante 8 semanas. Além disso, se você for um dos 1.500 alunos com melhor desempenho, receberá 20 semanas adicionais de formação com aulas on-line com um professor. Inscreva-se agora! Ah! Você não precisa ter um diploma universitário nem ser cliente do Banco Santander.
- https://app.santanderopenacademy.com/pt-BR/program/english-online-2026-2nd-call

Terá algum custo? –

Sem nenhum custo. Os cursos são financiados pelo Banco Santander. Se desejar, ao final do curso, você poderá se inscrever por conta própria para o próximo nível, mas essa opção é totalmente pessoal e você não terá nenhuma pressão para isso.



UniSENAI

Links interessantes

- https://www.oracle.com/br/java/
- https://academy.oracle.com/pt-br/oa-student.html
- https://www.jdoodle.com/online-java-compiler/
- https://glysns.gitbook.io/java-basico/

UniSENAI

Dúvidas? 😱

- **AVA** (https://ava.sesisenai.org.br/login/index.php)
- **E-mail** (matheus.krueger@edu.sc.senai.br)
- **WhatsApp** (https://wa.me/554733219647)

- **Monitoria**
  - Sábados, das 8h às 12h, na sala HUB.

"A persistência é o caminho do êxito."
Charles Chaplin